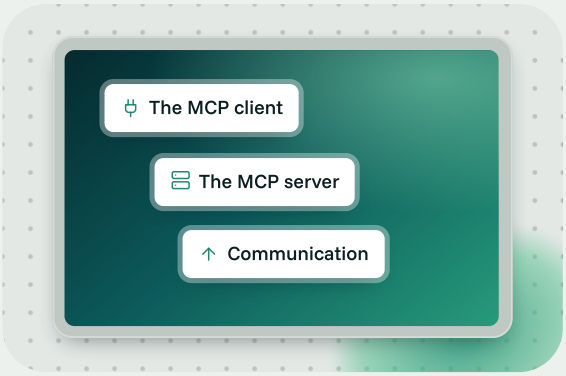
Last updated August 2026

Model Context Protocol (MCP)

The open standard for connecting AI models to external tools, data sources, and services through a universal interface.



13 min read · Beginner-friendly with advanced sections

Pascal Academy
AI fundamentals · Part 10 of 10

What Is MCP?

In the AI agents guide, we covered how agents use tools to interact with the world. An agent reads a database, calls an API, searches the web, or executes code. Each of these actions requires a connection between the model and an external system. Building those connections has historically been ad hoc. Every agent framework, model provider, and tool integration used its own format, its own authentication, and its own way of describing what the tool does.

The Model Context Protocol, or MCP, is an open standard that solves this fragmentation. It defines a common protocol for communication between AI applications and external data sources and tools. An easier way to think of it is as USB-C for AI integrations. Before USB-C, every device had its own connector, however, after USB-C, one cable can now work with everything. MCP aims to do the same for the connection between models and tools.

MCP was introduced by Anthropic in late 2024 and has since gained adoption across model providers, agent frameworks, and tool developers. The protocol is open source, meaning anyone can implement it without licensing restrictions, and it is designed to be model-agnostic, meaning it works with any LLM.

Key principle

MCP standardises the interface between AI models and external systems. A tool that exposes itself through MCP can be used by any MCP-compatible client, whether that client is Claude, ChatGPT, an open source agent framework, or a custom application. This eliminates the need to build a separate integration for every model-tool pair.

Why MCP Exists

Before MCP, connecting an AI model to an external tool required custom integration work. If you wanted your agent to read from a database, you wrote code that connected the model to the database, defined the tool's schema, and handled the communication format. If you then wanted the same agent to also read from a file system, you wrote another integration. If you switched model providers, you rewrote both integrations because the new provider used a different tool-calling format.

This N-by-M problem, where N models each need to support M tools, creates a combinatorial explosion of integrations. With 10 models and 100 tools, you need 1,000 separate integrations. MCP reduces this to N plus M. Each model implements the MCP client protocol once. Each tool implements the MCP server protocol once. Any model can then use any tool.

Before MCP	With MCP
Every model-tool pair needs a custom integration.	Each model implements MCP once. Each tool implements MCP once. Any model can use any tool.
Switching model providers requires rewriting all tool integrations.	Switching models requires no changes to tool integrations.
Tool definitions are specific to each agent framework.	Tool definitions are standardised and portable across frameworks.
Security and authentication are handled differently by each integration.	MCP defines standard security patterns for authentication and authorisation.

The practical benefit is reuse. A developer who builds an MCP server for a database makes that database accessible to every MCP-compatible AI application. A team that builds an MCP client in their agent gets access to every MCP server that exists, without writing additional integrations.

How MCP Works

MCP follows a client-server architecture. The MCP client runs inside the AI application, and the MCP server wraps the external tool or data source. The client and server communicate using a standard protocol based on JSON-RPC, a lightweight message format that sends structured requests and responses.

Pascal Academy

Pascal Academy's agent engineering course covers MCP server development, tool integration patterns, and production agent deployment. Reach out to learn about cohort programmes for your engineering team.

Ask about team training

The MCP Client

The client is the component that lives inside the AI application or agent framework. It manages connections to one or more MCP servers, discovers what tools and resources each server offers, and routes requests from the model to the appropriate server. When the model decides it needs to call a tool, the client sends the request to the MCP server that provides that tool, receives the response, and passes it back to the model.

The MCP Server

The server wraps an external system and exposes it through the MCP protocol. A server might wrap a database, a file system, an API like Slack or GitHub, a code execution environment, or any other system the model needs to interact with. The server advertises its capabilities, handles requests from the client, and returns results in a standardised format.

Communication

The client and server communicate over a transport layer. Two transports are commonly used. The first is stdio, where the client launches the server as a subprocess and communicates through standard input and output. This is simple and works well for local tools. The second is HTTP with Server-Sent Events, which allows the server to run on a remote machine. This is suitable for tools that need to be shared across multiple clients or that connect to cloud services.

Go deeper

The MCP protocol specification

Optional

MCP defines three types of capabilities that a server can expose. Tools are functions the model can call, like querying a database or sending a message. Each tool has a name, a description, and a schema describing its parameters. Resources are data the model can read, like files or database records. Each resource has a URI and a MIME type. Prompts are predefined prompt templates that the server offers, which can guide the model toward effective interactions. When a client connects to a server, the server advertises its capabilities, and the client makes them available to the model. The model then decides which tools to call and which resources to read based on the user's request.

MCP Components

Understanding MCP means understanding the three types of capabilities a server can expose and how the client uses them.

Component	What It Does	Example
Tools	Functions the model can call to perform actions or retrieve data. Each tool has a name, description, and parameter schema. The model decides when to call a tool based on the user's request and the tool's description.	A database query tool that accepts a SQL string and returns results. A file system tool that lists files in a directory. A Slack tool that sends a message to a channel.
Resources	Data the model can read, identified by a URI. Resources are read-only and provide context to the model. Unlike tools, resources are not called by the model. They are presented to the model as available context.	A file at file:///project/README.md. A database record at db://users/12345. A configuration file at config://app/settings.json.
Prompts	Predefined prompt templates that guide the model toward effective interactions with the server's tools and resources. Prompts can include placeholders that are filled in at runtime.	A code review prompt that instructs the model to read a file, analyse it, and suggest improvements using the server's file system tools.

The distinction between tools and resources matters. Tools are active, meaning the model calls them to perform an action or retrieve data. Resources are passive, meaning they are available context that the model can read. A database query tool lets the model run arbitrary SQL. A database resource presents a specific record to the model without requiring a tool call. The choice between exposing something as a tool or a resource depends on whether the model needs to actively request it or whether it should be available as context.

Building an MCP Server

Building an MCP server means wrapping an external system and exposing it through the protocol. The process involves defining the tools and resources the server offers, implementing the logic for each, and running the server on a transport.

Several SDKs are available for building MCP servers, including Python and TypeScript. The SDKs handle the protocol details, like message formatting and capability advertisement, so you can focus on the tool logic. A typical server defines its tools as functions, each with a name, description, and parameter schema. When a client sends a tool call request, the server executes the corresponding function and returns the result.

Design Considerations

When building an MCP server, several design decisions affect usability and safety. Tool descriptions should be clear and specific, because the model uses them to decide when to call each tool. A vague description like search data is less effective than search the product catalogue by name, category, or SKU, returning matching products with their prices and availability. The model needs enough detail to understand what the tool does and when to use it.

Parameter schemas should be precise. If a parameter accepts an enum, list the valid values. If a parameter is required, mark it as required. If a parameter has a default, specify it. The more information the schema provides, the less likely the model is to call the tool with incorrect parameters.

Security is a primary concern. An MCP server that wraps a database should not let the model run arbitrary DROP TABLE statements. Read-only tools should be clearly separated from write tools. Authentication should be required for sensitive operations. The server should validate all inputs and return clear error messages when parameters are invalid or unauthorised.

Go deeper

The lifecycle of an MCP tool call

Optional

When a model decides to call a tool, the flow is as follows. The client sends the user's message to the model along with the list of available tools and their schemas. The model processes the message and decides to call a tool, producing a tool call request with the tool name and parameters. The client receives this request and routes it to the MCP server that provides the tool. The server executes the tool function with the given parameters and returns the result. The client passes the result back to the model as a tool response. The model uses the response to generate its answer or to decide whether another tool call is needed. This cycle can repeat multiple times for complex tasks, similar to the ReAct loop covered in the AI agents guide.

MCP and Agents

MCP and agents are complementary. As covered in the AI agents guide, an agent uses a model to plan and execute multi-step workflows, calling tools along the way. MCP provides the standardised interface through which those tool calls happen.

Without MCP, an agent framework needs custom integrations for every tool it supports. The framework defines its own tool-calling format, and developers write adapters to connect external systems. With MCP, the framework only needs to implement the MCP client protocol. Every MCP server becomes available as a tool source, and the framework does not need to know the details of how each tool works.

This separation of concerns is valuable. Agent developers can focus on planning, reasoning, and orchestration without reimplementing tool integrations for each new data source. Tool developers can build MCP servers without knowing which agent frameworks will use them. The protocol acts as a clean boundary between the two layers.

Pascal Academy

Pascal Academy's agent engineering course covers MCP integration, tool design patterns, and production agent architecture. Reach out to learn about team training options.

Ask about team training

MCP vs Function Calling

Function calling is a capability built into modern language models. The model is given a list of functions with their parameter schemas, and it can decide to call a function by producing a structured output with the function name and arguments. The application executes the function and returns the result to the model.

MCP and function calling serve different layers of the stack. Function calling is a model capability, meaning the model can produce structured tool-call requests. MCP is a protocol, meaning it defines how those requests are routed to external systems and how results are returned. The two work together. The model uses function calling to decide which tool to call and with what parameters. MCP transports that request to the appropriate server, executes it, and returns the result.

Function Calling	MCP
A model capability. The model produces structured tool-call requests.	A protocol for routing those requests to external systems.
Specific to each model provider. OpenAI, Anthropic, and Google each have their own format.	Model-agnostic. Works with any model that supports tool calling.
Defines how the model expresses a tool call, not how the call is executed.	Defines how the call is transported, executed, and the result returned.
The application is responsible for connecting each tool call to the right external system.	The protocol handles routing to the right server and returning the result.

In practice, an MCP-compatible application uses function calling internally. The model produces a tool-call request using its native function-calling format. The application translates this into an MCP request and sends it to the server. The distinction is that MCP standardises the layer between the model and the tool, so the application does not need custom code for each tool.

Security and Trust

MCP servers give AI models access to external systems, which means they also introduce security considerations. A model that can query a database can potentially access sensitive data. A model that can send messages can potentially send unauthorised communications. A model that can execute code can potentially run harmful commands.

Several security practices are important when deploying MCP. Least privilege means each server should expose only the tools and resources the model needs, with the minimum permissions required. A database server for a read-only Q and A system should not expose write or delete tools. Authentication means the server should verify the identity of the client and enforce access controls. Input validation means the server should validate all parameters before executing a tool, rejecting requests that contain invalid or potentially harmful input.

User consent is another consideration. For actions that have real-world consequences, like sending an email or modifying a database record, the application should ask the user for confirmation before the server executes the tool. This is human-in-the-loop, as covered in the AI agents guide. MCP itself does not enforce these practices. It is a protocol, not a security framework. The responsibility for secure deployment falls on the developers building and configuring the servers.

The MCP Ecosystem

Since its introduction, MCP has grown a substantial ecosystem of servers, clients, and tools. Several categories have emerged.

Reference Servers

Anthropic and the community have built reference MCP servers for common integrations. These include servers for file system access, database querying, web search, Git repositories, Slack, GitHub, and Google Drive. These servers can be used directly or adapted for specific needs. They serve as examples of how to build well-structured MCP servers.

Client Support

MCP is supported by several AI applications and agent frameworks. Claude Desktop was the first MCP client, allowing users to connect local tools through MCP servers. Other applications, including open source agent frameworks and development tools, have added MCP client support. As adoption grows, the value of the protocol increases, because each new client gains access to every existing server and each new server becomes available to every existing client.

Custom Servers

Organisations build custom MCP servers to expose internal systems to AI applications. A company might build an MCP server for its internal wiki, its CRM, its ticketing system, or its deployment pipeline. Once built, these servers can be used by any MCP-compatible client, whether that is a local assistant, a cloud-based agent, or a custom application.

The Future of MCP

MCP is still early in its adoption curve. The protocol defines the basic mechanics of tool discovery, invocation, and result return, but several areas are evolving.

Remote deployment is one direction. Currently, many MCP servers run locally through stdio transport, which limits them to single-user, single-machine setups. Remote MCP servers running over HTTP would enable shared tool access across teams and organisations. This raises additional security considerations, including authentication, encryption, and access control, which the protocol is beginning to address.

Standardisation across model providers is another direction. While MCP is model-agnostic, the way models use MCP-exposed tools still depends on their native function-calling format. Further standardisation of the tool-calling layer would reduce the remaining differences between model providers.

The broader question is whether MCP becomes the universal standard for AI-tool integration, or whether competing protocols fragment the ecosystem. The open source nature of MCP, combined with growing adoption from both model providers and tool developers, gives it a strong position. But standards succeed when they reach critical mass, and MCP is still building toward that point.

Summary

The Model Context Protocol is an open standard for connecting AI models to external tools, data sources, and services. It uses a client-server architecture where the client lives inside the AI application and the server wraps an external system. Servers expose three types of capabilities. Tools are functions the model can call. Resources are data the model can read. Prompts are predefined templates that guide the model. MCP solves the N-by-M integration problem by standardising the interface, so any model can use any tool through a single protocol. It complements function calling, which is the model's ability to produce tool-call requests, by handling the transport and execution layer. Security is a primary concern when deploying MCP servers, and practices like least privilege, authentication, input validation, and human-in-the-loop confirmation are essential. The MCP ecosystem is growing, with reference servers for common integrations and custom servers for internal systems. The protocol is still early in its adoption curve, with ongoing development in remote deployment, cross-provider standardisation, and security.

About this guide

This guide is part of Pascal Academy's AI Fundamentals series, covering LLMs, prompt engineering, RAG, agents, context windows, knowledge graphs, hallucinations, reasoning models, embeddings, and MCP. The full series and hands-on courses are available at Pascal Academy. For teams looking to upskill, we offer custom cohort programmes tailored to your stack and use cases.

Frequently asked questions

What is the Model Context Protocol (MCP)?

MCP is an open standard that defines how AI models connect to external data sources and tools. It provides a common interface so that an AI model can retrieve information from

databases, file systems, and APIs without requiring custom integrations for each source.

Why is MCP important for AI in finance?

Finance teams work with data from Bloomberg, Capital IQ, FactSet, internal CRM systems, and filings. Without a standard protocol, connecting an AI model to each source requires a custom integration. MCP simplifies this by providing a common interface, so one connection can reach multiple data sources.

How does Pascal implement MCP for institutional finance?

Pascal is MCP-native, meaning its agents connect to your existing data providers through the Model Context Protocol. Bloomberg, SharePoint, Snowflake, and your CRM connect through one MCP Hub. This means Pascal's agents can pull from all your data sources without custom pipelines, and citations trace back to the source line regardless of where the data originated.

What is the difference between MCP and an API?

An API connects a specific application to another specific application with a custom interface. MCP is a standard protocol that defines how any AI model connects to any data source. APIs are point-to-point connections. MCP provides a common layer so one model can reach many sources through a consistent interface.